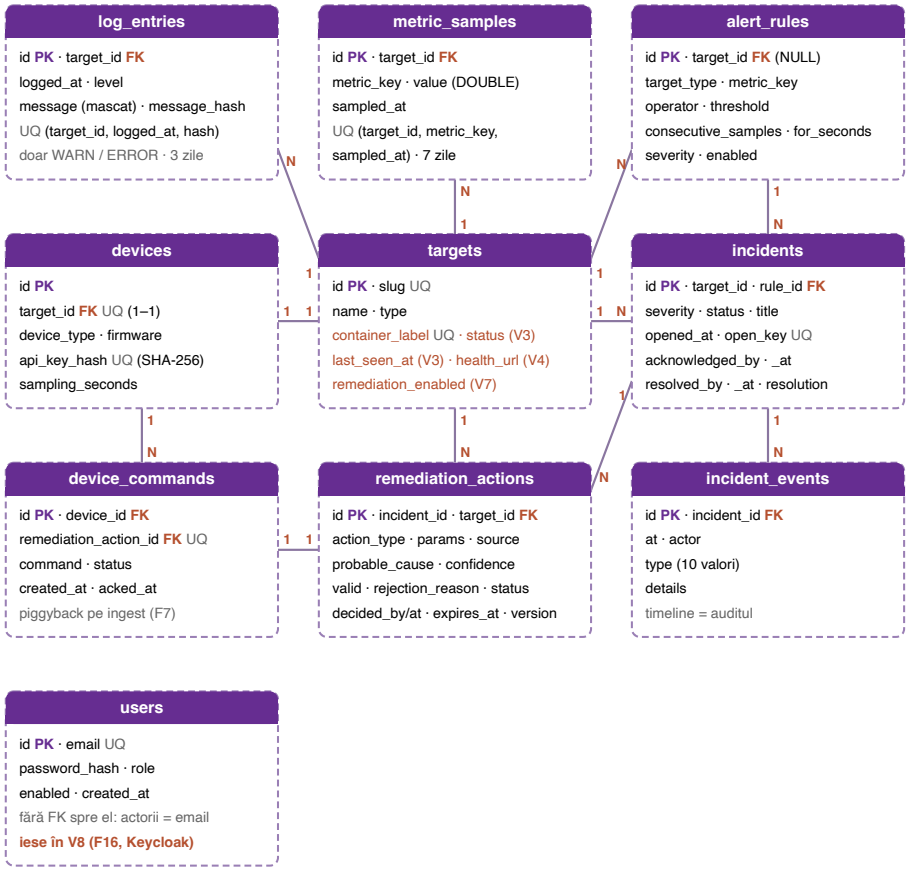
Structura bazei de date

MySQL 8.4 (`utf8mb4`) · Flyway · 10 tabele (9 la final: `users` iese în F16), toate în baza lui `netpilot-api` · se construiesc feature cu feature, după `PLAN.pdf` · Vlad

Diagrama



PK cheie primară, `BIGINT` auto-increment · **FK** cheie străină · **UQ** valoare unică · **NN** obligatoriu (`NOT NULL`) · chenar punctat = tabel nou: toate sunt noi, repo-ul pornește fără cod · eticheta de lângă nume = feature-ul și migrarea care creează sau schimbă tabelul · **coloană cărămizie** = adăugată de o migrare ulterioară celei care a creat tabelul · actorii (`*_by`, `actor`) sunt text, emailul din token sau `system`: niciun FK spre `users`, ca tabelul să poată ieși în F16.

Tabelele

targets F1 · V1 · F3 · V3 · F4 · V4 · F7 · V7

Ce monitorizează NetPilot: un container al tău sau un dispozitiv. Un singur tabel, fără ierarhie JPA.

coloană	tip	reguli
id	BIGINT	PK
slug	VARCHAR(50)	NN, UQ · a–z 0–9 –, 3–50
name	VARCHAR(100)	NN
type	VARCHAR(10)	NN · CONTAINER DEVICE
created_at	DATETIME(6)	NN
container_label	VARCHAR(50)	UQ · valoarea etichetei netpilot.target, doar la CONTAINER · V3
status	VARCHAR(10)	NN, implicit UNKNOWN · UP DEGRADED DOWN · V3
last_seen_at	DATETIME(6)	ultima citire / ultimul POST · V3
health_url	VARCHAR(255)	doar http(s)://, doar la CONTAINER · V4
remediation_enabled	BOOLEAN	NN, implicit false · regula 2 a validatorului · V7

Identitatea containerului e eticheta, nu ID-ul Docker (compose recreează containerul la deploy). Detaliile unui dispozitiv stau în `devices`. `SINGLE_TABLE` cu subclase e respins: pattern forțat pentru două tipuri. FK-urile spre `targets` sunt pe RESTRICT: ștergerea unei ținte cu istoric → 409.

users F2 · V2 · F16 · V8

Conturile, până le preia Keycloak. Fără înregistrare publică.

coloană	tip	reguli
id	BIGINT	PK
email	VARCHAR(255)	NN, UQ
password_hash	VARCHAR(100)	NN · BCrypt
role	VARCHAR(10)	NN · VIEWER OPERATOR ADMIN AGENT
enabled	BOOLEAN	NN, implicit true · DELETE = dezactivare
created_at	DATETIME(6)	NN

Permisiunile rezultă din `role`, dintr-o singură hartă în cod (`RolePermissions`). Adminul nu vine din migrare, ci din `AdminSeeder` (`ADMIN_EMAIL` / `ADMIN_PASSWORD`). **V8 șterge tabelul**: din F16 conturile și rolurile stau în Keycloak.

metric_samples F3 · V3

Un eșantion al unei ținte: datele de domeniu ale lui NetPilot.

coloană	tip	reguli
id	BIGINT	PK
target_id	BIGINT	NN, FK → targets.id, RESTRICT
metric_key	VARCHAR(30)	NN · setul permis per tip, în cod
sampled_at	DATETIME(6)	NN · începutul ciclului / ora serverului la ingest
value	DOUBLE	NN

Chei: containere `cpu_percent` `mem_percent` `restart_count` `running` (F3) `health_up` `http_latency_ms` (F4); dispozitive `temperature_c` `battery_percent` `rssi_dbm` (F5). UQ `(target_id, metric_key, sampled_at)`: același eșantion nu intră de două ori și servește interogarea pe interval; INDEX `(sampled_at)` pentru retenție. În entitate `double value` (DOUBLE ↔ `double` trece de `validate`). Retenția 7 zile: pe EC2 158 400 rânduri/zi, cel mult 8 zile înaintea jobului = 1 267 200; plafonul din analiză 233 280/zi → cel mult 1 866 240. Ultima oră brut, 24 h pe găleți de 5 minute, fără tabel de agregare.

log_entries F4 · V4

O linie importantă din logul unui container, mascată la colectare.

coloană	tip	reguli
id	BIGINT	PK
target_id	BIGINT	NN, FK → targets.id, RESTRICT
logged_at	DATETIME(6)	NN · din timestamps=1
level	VARCHAR(5)	NN · WARN ERROR (Exception fără nivel → ERROR)
message	VARCHAR(1000)	NN · mascat (password=, secret, Bearer), tăiat la 1000
message_hash	CHAR(64)	NN · SHA-256 al mesajului mascat

UQ `(target_id, logged_at, message_hash)`: `since` din Docker e inclusiv, deci ultima linie revine la ciclul următor; colectorul o sare (timestamp ≤ ultimul salvat), iar UQ-ul oprește orice dublură rămasă. Tot el servește `/logs` și `getRecentLogs` (prefixul `target_id, logged_at`); INDEX `(logged_at)` pentru retenția de 3 zile. Logurile platformei (ELK, F15) nu intră aici; agentul și Claude văd doar acest tabel.

devices F5 · V5

Detaliile unei ținte de tip `DEVICE`, 1–1 cu ea.

coloană	tip	reguli
id	BIGINT	PK
target_id	BIGINT	NN, UQ, FK → targets.id
device_type	VARCHAR(30)	NN
api_key_hash	CHAR(64)	NN, UQ · SHA-256 hex al cheii
firmware	VARCHAR(30)	
sampling_seconds	INT	NN, implicit 10 · între 5 și 300

Creat în aceeași tranzacție cu ținta lui. Cheia în clar apare o singură dată, în răspunsul la creare / rotire. La ingest, ținta se ia din cheie, niciodată din corp. `sampling_seconds` se schimbă la confirmarea unui `SET_SAMPLING_INTERVAL` (F7).

alert_rules F6 · V6

O regulă de prag sau de lipsă de date; cele 6 implicite vin din migrare.

coloană	tip	reguli
id	BIGINT	PK
target_type	VARCHAR(10)	NN · CONTAINER DEVICE
target_id	BIGINT	FK → targets.id · NULL = toate țintele tipului
metric_key	VARCHAR(30)	NN · NO_DATA la regula specială
operator	VARCHAR(2)	GT LT EQ · NULL la NO_DATA
threshold	DOUBLE	NULL la NO_DATA
consecutive_samples	INT	NN, între 1 și 20
for_seconds	INT	doar la NO_DATA
severity	VARCHAR(10)	NN · WARNING CRITICAL
enabled	BOOLEAN	NN

INSERT-urile din V6: `CONTAINER running EQ 0` ×2 CRITICAL · `CONTAINER health_up EQ 0` ×2 CRITICAL · `CONTAINER mem_percent GT 90` ×10 WARNING · `DEVICE temperature_c GT 70` ×3 CRITICAL · `DEVICE battery_percent LT 15` ×1 WARNING · `DEVICE NO_DATA 60 s` CRITICAL.

incidents F6 · V6

O încălcare a unei reguli pe o țintă, de la deschidere la rezolvare.

coloană	tip	reguli
id	BIGINT	PK
target_id	BIGINT	NN, FK → targets.id, RESTRICT
rule_id	BIGINT	NN, FK → alert_rules.id
severity	VARCHAR(10)	NN · copiată din regulă la deschidere
status	VARCHAR(15)	NN · OPEN ACKNOWLEDGED RESOLVED (final)
title	VARCHAR(200)	NN
opened_at	DATETIME(6)	NN
acknowledged_by	VARCHAR(255)	emailul omului
acknowledged_at	DATETIME(6)	
resolved_by	VARCHAR(255)	emailul omului sau system
resolved_at	DATETIME(6)	
resolution	VARCHAR(10)	AUTO MANUAL
open_key	VARCHAR(40)	UQ · <rule_id>:<target_id> cât e deschis, NULL la rezolvare

`open_key` unic = un singur incident deschis per regulă și țintă, garantat de bază, nu doar de cod (MySQL nu are index parțial, de aici NULL la rezolvare). INDEX `(status, opened_at)` pentru `/incidents?status=` și `listOpenIncidents`.

incident_events F6 · V6

Timeline-ul incidentului; tot el e auditul.

coloană	tip	reguli
id	BIGINT	PK
incident_id	BIGINT	NN, FK → incidents.id
at	DATETIME(6)	NN
actor	VARCHAR(255)	NN · emailul sau system
type	VARCHAR(20)	NN · OPENED ACKNOWLEDGED AGENT_STARTED PROPOSED APPROVED REJECTED EXECUTED FAILED RESOLVED NOTE
details	VARCHAR(1000)	

Se scrie în aceeași tranzacție cu schimbarea de stare și nu se modifică / șterge niciodată. INDEX `(incident_id, at)`. Citit prin `/incidents/{id}/events`, cu `AUDIT_VIEW` (F7). Latența din comparația F11 = `PROPOSED.at − OPENED.at`.

remediation_actions F7 · V7

O acțiune pe un incident: propusă de om sau de agent, validată, aprobată de un om, executată și verificată.

coloană	tip	reguli
id	BIGINT	PK · din F13 și cheia de idempotență Kafka
incident_id	BIGINT	NN, FK → incidents.id
target_id	BIGINT	NN, FK → targets.id · scris mereu = ținta incidentului
requested_target_id	BIGINT	fără FK · ținta cerută de propunător, dacă a dat una; ținta incidentului (chiar inexistentă) → TARGET_MISMATCH (regula 1) · V7
action_type	VARCHAR(25)	NN · RESTART_CONTAINEF REBOOT_DEVICE SET_SAMPLING_INTERVAI NO_ACTION
params	JSON	{"seconds": 5–300} do la SET_SAMPLING_INTERVAI
proposed_by	VARCHAR(255)	NN · emailul
source	VARCHAR(12)	NN · HUMAN AGENT_RULES AGENT_LLM
probable_cause	VARCHAR(500)	
confidence	DOUBLE	între 0 și 1
evidence	VARCHAR(2000)	
valid	BOOLEAN	NN · rezultatul validatorului propunere
rejection_reason	VARCHAR(30)	TARGET_MISMATCH ACTION_NOT_ALLOWED REMEDIATION_DISABLED INVALID_PARAMS COOLDOWN_ACTIVE TARGET_NOT_LABELED INCIDENT_RESOLVED
status	VARCHAR(25)	NN · PROPOSED REJECTED_BY_VALIDATI(APPROVED REJECTED EXPIRED EXECUTING SUCCEEDED FAILED
decided_by	VARCHAR(255)	emailul omului care a aprob / respins
decided_at	DATETIME(6)	
executed_at	DATETIME(6)	baza cooldown-ului
result_message	VARCHAR(500)	
expires_at	DATETIME(6)	NN · propunerea + 15 min
version	BIGINT	NN · @Version
open_proposal_key	BIGINT	UQ · = incident_id cât e PROPOSED, NULL după
created_at	DATETIME(6)	NN

În entitate: `String params` cu `@JdbcTypeCode(SqlTypes.JSON)`. Trecerile `PROPOSED → APPROVED` și `APPROVED → EXECUTING` prin UPDATE condiționat pe status (1 rând = continuă, 0 = 409 / duplicat sărit). Un singur `PROPOSED` deschis per incident, garantat de UQ pe `open_proposal_key` (ca `open_key` la incidente). Un `targetId` inexistent nu lovește FK-ul: rămâne în `requested_target_id`, iar propunerea iese `REJECTED_BY_VALIDATION`. INDEX `(incident_id, status)` și `(target_id, action_type, executed_at)` pentru cooldown (1 restart / 10 min, 3 / oră). `source` e baza comparației reguli vs Claude (F11).

`device_commands` F7 · V7

O comandă aprobată pentru un dispozitiv, livrată în răspunsul la telemetrie.

coloană	tip	reguli
`id`	`BIGINT`	PK
`device_id`	`BIGINT`	NN, FK → `devices.id`
`remediation_action_id`	`BIGINT`	NN, UQ, FK → `remediation_actions.id`
`command`	`VARCHAR(30)`	NN · `REBOOT_DEVICE` `SET_SAMPLING_INTERVAL`
`status`	`VARCHAR(10)`	NN · `PENDING` `DELIVERED` `ACKED`
`created_at`	`DATETIME(6)`	NN
`acked_at`	`DATETIME(6)`	fără confirmare în 60 s → acțiunea `FAILED`

UQ pe `remediation_action_id` = o acțiune dă cel mult o comandă, chiar dacă aprobarea ajunge de două ori (idempotența din F13). Secundele unui `SET_SAMPLING_INTERVAL` se iau din `remediation_actions.params`.

Migrările, feature cu feature. `ddl-auto: validate` verifică la fiecare pornire că tabelele se potrivesc cu entitățile. Deci o migrare intră **în același commit** cu entitatea ei, în `netpilot-api/src/main/resources/db/migration/`, iar una care a rulat pe EC2 nu se mai editează. `netpilot-db/` păstrează doar scripturile de inițializare / seed și documentația schemei, nu migrări.

migrare	feature	ce face
`V1__targets`	F1	`targets`, exact entitatea `Target` de la finalul F1 (`id`, `slug`, `name`, `type`, `created_at`); InnoDB, `utf8mb4`
`V2__users`	F2	`users`; adminul nu vine din migrare, ci din `AdminSeeder`
`V3__metric_samples`	F3	`metric_samples` cu UQ `(target_id, metric_key, sampled_at)` și INDEX `(sampled_at)`; pe `targets`: `container_label` (UQ), `status`, `last_seen_at`
`V4__logs_health`	F4	`log_entries` cu UQ `(target_id, logged_at, message_hash)` și INDEX `(logged_at)`; `targets.health_url`
`V5__devices`	F5	`devices`, 1–1 cu `targets` (FK UQ), `api_key_hash` UQ
`V6__incidents`	F6	`alert_rules` + INSERT-urile celor 6 reguli implicite; `incidents` cu `open_key` UQ și INDEX `(status, opened_at)`; `incident_events` cu INDEX `(incident_id, at)`
`V7__remediation`	F7	`remediation_actions` cu `requested_target_id` (fără FK), UQ pe `open_proposal_key` și indexurile de cooldown; `device_commands` cu UQ pe `remediation_action_id`; `targets.remediation_enabled` (implicit `false`)
`V8__drop_users`	F16	`users` iese: conturile le ține Keycloak; niciun FK nu pointa spre el

F1b, F8–F15 și F17 nu schimbă schema. Fără tabele: `netpilot-simulator` (F5), `netpilot-agent` (F10) și `netpilot-remediator` (F14) n-au bază de date: simulatorul e un sistem extern, agentul vede totul prin MCP, remediatorul doar consumă și publică în Kafka. Idempotența din Kafka (F13) stă în UPDATE-ul condiționat pe `remediation_actions.status` și în UQ-ul din `device_commands`, fără tabel separat. Keycloak (F16) are Postgres-ul lui, în afara acestei scheme. Prometheus (F6) nu conține nicio serie per țintă: valorile țintelor stau doar aici.